AI-Powered Agriculture Crop Advisory Assistant

Architecture Diagram & System Design

1. High-Level System Architecture

The system follows a deterministic, multi-layer architecture that decouples user interaction, AI reasoning, retrieval, machine learning inference, and output validation into independent, composable modules.

End-to-End Request Flow:

Stage	Component	Technology	Output
1. Input	Guardrail Agent	Rule-based NLP + Regex	Sanitized query
2. Classify	Query Classifier Agent	Keyword + Pattern Matching	Structured parameters
3. Retrieve	Hybrid RAG Engine	FAISS + TF-IDF + RRF	Top-K cases
4. Re-rank	Reranker	Agronomic multi-factor scoring	Ranked case list
5. Recommend	Recommendation Agent	Historical evidence fusion	Crop candidates
6. Risk	Risk Analysis Agent	Rule + stat thresholds	Risk flags
7. Yield	Yield Prediction Agent	RandomForestRegressor	Yield + 80% CI
8. Impact	Impact Analysis Agent	Feature importance correlation	Limiting factors
9. Validate	LLM Judge Agent	Deterministic scorer	Grounded advisory
10. Output	FastAPI Response	JSON + Formatted Advisory	Section 45 output

2. Data Layer

- **Vector Store:** FAISS IndexFlatIP — 1,500 agricultural case embeddings (384-dim SVD-reduced TF-IDF)
- **Keyword Index:** TF-IDF Inverted Index — sparse BM25-style retrieval for exact agronomic terms
- **ML Models:** RandomForestClassifier (suitability) + RandomForestRegressor (yield) — scikit-learn
- **Feedback Store:** JSON-based persistent feedback log at data/feedback_store.json
- **Evaluation Reports:** JSON + Markdown evaluation benchmarks at evaluation/reports/

3. Infrastructure & Deployment

- **Backend:** FastAPI (ASGI) served via Uvicorn on port 8000
- **Frontend:** Streamlit SPA on port 8501, calling REST API
- **Static UI:** Vanilla HTML/CSS/JS SPA also served at port 8000/static
- **Docker:** Optional containerization via Dockerfile + docker-compose.yml
- **Environment:** Python 3.13, Windows / Linux / macOS compatible

4. Component Dependency Map

User Query → FastAPI /api/query → GuardrailAgent → QueryClassifierAgent → HybridRAGRetriever [FAISS + TF-IDF] → Reranker → RecommendationAgent → RiskAnalysisAgent → YieldPredictionAgent [RandomForest] → ImpactAnalysisAgent → LLMJudgeValidator → FormattedAdvisory → Client

5. Security & Guardrails Architecture

- Input Guardrail: Regex-based injection detection, domain scope classifier, numeric bound validation
- Output Guardrail: Section 45 disclaimer injection, hallucination suppression
- LLM Judge: 4-dimension offline-safe validator (Relevance, Groundedness, Completeness, Safety)
- All API routes protected by Pydantic v2 strict type validation